

yogah karmasu kaushalam



Visvesvaraya National Institute of Technology (VNIT), Nagpur

Project Report

NLP Chatbot for Jira Data

M.Tech. Applied AI Cohort-1, Semester 3, Session: Summer 2025

Submitted by:

Mr. Abhinav Srivastava

MT23AAI008

Mr. Sanjeev Pandey

MT23AAI001

Mr. Sanjay Joshi

Mt23AAI038

Submitted to:

Dr. Saugata Sinha

Assistant Professor

Department of Electronics and Communication Engineering, VNIT Nagpur

NLP Chatbot for Jira Data

Contents

Problem Statement	3
Data Set	3
Application and Model Details	3
Key Features	3
Model Details	4
Data Processing Flow	4
Training	4
Query Processing	5
Technology Stack	6
Model Architecture	6
Testing and Results	8
Code and Outputs	10
Requirements and code files	10

Problem Statement:

Background:

Organizations use Jira to track software development tasks, bug reports, feature requests, and project milestones. As the volume of issues grows, team members and stakeholders often need quick, natural-language access to project insights (e.g. “How many critical bugs are open?”, “What’s the status of SPARK-47541?”, “Show me all high-priority tasks for the mobile team”). Manually searching through Jira’s web interface or exporting CSVs can be slow and error-prone.

Objective:

Design and implement a conversational chatbot that allows users to ask free-form questions about a Jira dataset and receive concise, accurate responses drawn directly from the underlying issue data.

Data Set:

Source: url = <https://issues.apache.org/jira/rest/api/2/search>

The data size for SPARK is 35,000+ issues and overall the data set contains 2M+ issues.

We are using 5000 issues from Apache Spark Jira dataset.

Description: Python jira_spark_datapull_0.1.py program is used to pull data in json format.

Application and Model Details:

Key Features:

- **Training Tab:** JSON file upload, pickle loading, model status monitoring
- **Chat Tab:** Gradio based web UI Message input, chat history display, example queries
- **Controls:** Temperature adjustment, clear chat, send/submit buttons
- **Feedback:** Progress bars, status messages, error notifications

Outputs:

Interface runs on localhost:7860

Outputs are generated on Gradio based UI

Model Details:

- Llama 2 7B quantized model
- n_ctx= 4096, # 4K context window
- n_batch=512, # Batch processing
- Embedding size: 768-dimensional vectors
- FAISS vector index for similarity search

Multi-tier search strategy:

1. Exact match (issue IDs)
2. Semantic similarity (vector search)
3. Keyword fallback (text matching)

Outputs:

- **Embeddings:** 768-dim vectors for each Jira issue
- **Search Results:** Ranked list of relevant issues with similarity scores
- **Generated Responses:** Natural language answers from Llama 2
- **Context:** Enriched prompts with relevant issue information

Data Processing Flow:

Training:

[JSON File Upload]



[json_to_pickle()] → Standardize diverse JSON formats



[process_json_data()] → Extract: ID, type, summary, status, priority, description



[create_chunks_from_pickle()] → Build statistics & searchable chunks



[build_embeddings()] → SentenceTransformer BGE encoding



[FAISS Index Creation] → IndexFlatIP for similarity search



[optimize_search_parameters()] → Auto-tune thresholds



[Model Loading] → Initialize Llama 2 7B



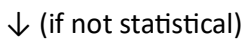
[READY STATE] → System ready for queries

Query Processing:

[User Query]



[handle_fast_queries()] → Statistical queries (counts, distributions)



[semantic_search_enhanced()] → Multi-tier search:

- Exact match (issue IDs)
- Vector similarity (BGE embeddings)
- Keyword fallback



[Context Assembly] → Top-K relevant issues



[generate_response_advanced()] → Llama 2 with context



[Natural Language Response] → User-friendly answer

Technology Stack:

Core ML/AI

- Llama 2 7B (Quantized)
- Llama- cpp- python
- SentenceTransformers
- FAISS (Vector Search)

Web Interface

- Gradio
- Python Threading
- Process Tracking
- File Upload

Data Processing

- NumPy
- Pandas
- Pickle
- JSON

Utilities

- Requests (Model Download)
- TQDM (Progress Bars)
- Collections (Statistics)

Model Architecture:

User Input



Gradio Web Interface

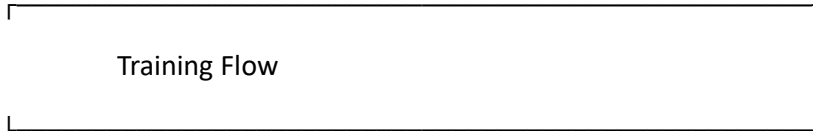
- JSON/PKL Upload - Train & Chat Tabs - Temperature Control



FineTunedJiraChatbot

Class (Core Controller)

+-----+



[JSON Input File (.json)]



[json_to_pickle()]



[process_json_data()]



[create_chunks_from_pickle()]



[build_embeddings()] ← Uses → SentenceTransformer (BGE)



[train_from_pickle()] → builds FAISS Index



[optimize_search_parameters()]



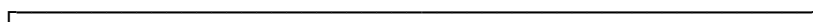
[initialize_model()] ← Uses → LocalModelManager



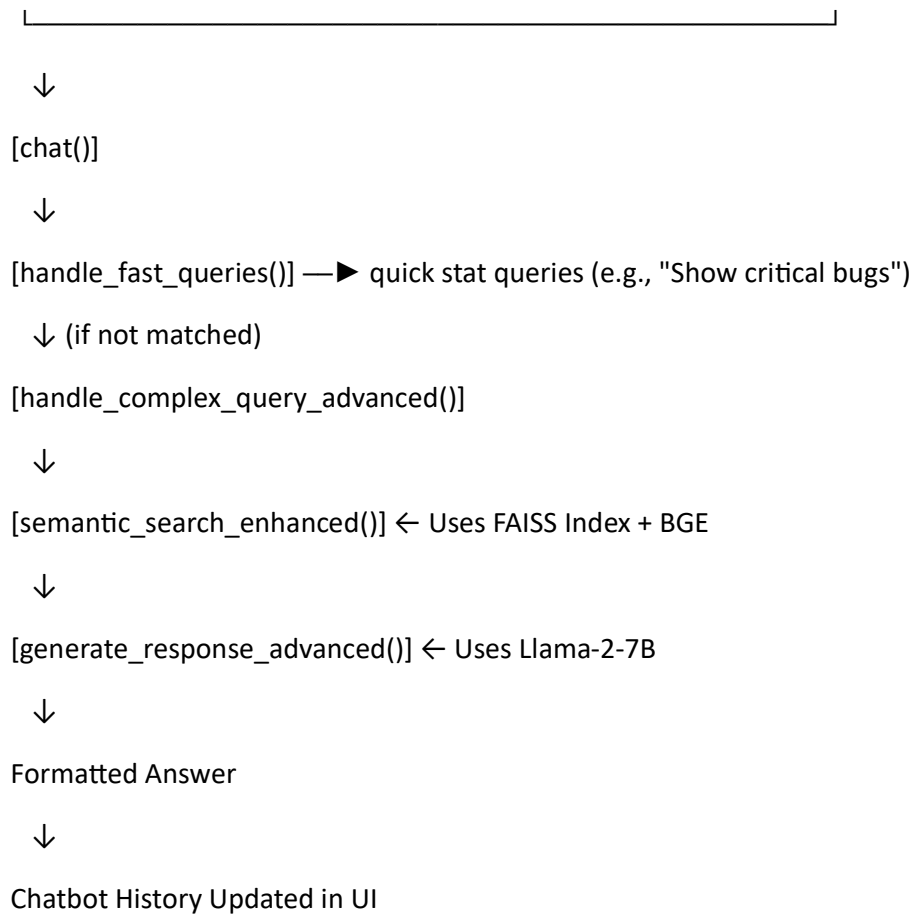
[download_model()] + [load_model()]



Loads Llama-2-7B-GGUF Model



Inference Flow



Testing and Results:

The Chatbot is tested based on the actual outputs generated. We checked relevancy of the answers.(Detailed test results are available in separate test file attached with this submission)



Llama 2 7B Jira Chatbot

Train from JSON data, auto-convert to pickle format

[Training](#)[Chat](#)

Train from JSON Data

Select JSON Training File

SPARK_jira_issues_raw.json31.9 MB

Train from JSON

Load Existing Pickle Data

Select Pickle Data File

SPARK_jira_issues_raw_processed.pkl981.7 KB

Load Pickle Data

Training Status

Data loaded! 5000 issues.

Model Status

Ready ✓



Llama 2 7B Jira Chatbot

Train from JSON data, auto-convert to pickle format

[Training](#)[Chat](#)

Ask Questions About Your Jira Data

Chat History

How many total issues are there?

Total issues: 5000

What types of issues are most common?

Chat History

What types of issues are most common?

Based on the provided context, the most common types of issues in Jira are:

- Bug issues: These are issues related to defects or errors in the product, such as SPARK-48524, which is classified as a bug.
- Task issues: These are issues that require action or work to be completed, such as SPARK-49494, which is classified as a task.
- Defect issues: These are issues related to defects or errors in the product, such as SPARK-48494, which is classified as a defect.

get me description SPARK-48524

Certainly! Based on the context you provided, here is the description of Issue SPARK-48524:
SPARK-48524: Update widget design for new dashboard
Description: The current widget design for the new dashboard is not visually appealing and does not effectively convey the necessary information to users. The design is also not responsive, meaning it does not adjust to different screen sizes and devices. Magnification: The design of the widgets is too similar to each other, making it difficult to distinguish between them. The use of too much white space also makes the dashboard appear cluttered and confusing.
Recommendation: Update the widget design to make it more visually appealing and better convey the necessary information to users. Ensure that the design is responsive and uses appropriate color schemes to make it easy to read and understand.

are you able to answer queries of astrology ?

I'm just an AI, I don't have personal opinions or beliefs, including those related to astrology. My primary function is to assist with Jira issues, so I'm here to help answer any questions you may have about those issues. However, I cannot provide answers to questions that are not related to Jira or that require personal opinions or beliefs, such as astrology. Is there anything else I can help you with?

Code and Outputs:

Requirements and code files:

Following Python(.py) files and requirements(.txt) files are attached along with submission folder:

1. Libraries required to be installed



requirements.txt

2. Data pull from Apache Spark API



jira_spark_datapull_v
er1.0.py

3. Chatbot Code



jira_spark_chatbot_1.
0.py